RadeonSort

Robust Multi-Class Robotic Sorting with Failure Detection and Automatic Recovery

Challenge	AMD Radeon Hackathon 2026
Track	Track 3 - Physical AI
Team	520lake (solo developer)
Platform	AMD Radeon GPU, ROCm 7.2, Genesis 1.2.3
Report date	29 July 2026

30 randomized trials | 86.67% first-attempt success | 96.67% final success after recovery

1. Executive Summary

RadeonSort is a Physical AI simulation system for robust robotic sorting in small warehouse environments. A Franka Panda arm sorts three YCB objects - banana, lemon, and plum - into separate physical bins. Unlike an open-loop pick-and-place demonstration, the system verifies the result of every attempt. If an object does not reach the target bin, RadeonSort returns it to a recovery station, changes the grasp and motion policy, and retries automatically.

The project runs on a Radeon Cloud AMD GPU with ROCm and the Genesis `gs.amdgpu` backend. The submitted benchmark contains 30 seeded randomized trials with +/-0.025 m position jitter and at most two retries. First-attempt success was 86.67%; closed-loop recovery increased final success to 96.67% (29/30).

Application and target users

The target application is low-volume warehouse or laboratory sorting where object pose and surface friction vary, but a full industrial perception and planning stack may be too expensive. Target users include robotics students, simulation engineers, small automation teams, and researchers evaluating failure-aware manipulation policies.

Project scope

- Three-class task routing to three physically bounded target bins.
- Scripted inverse-kinematics pick-and-place using the Track 3 reference scene.
- Closed-loop spatial verification after every attempt.
- Adaptive recovery through yaw, position, force, height, and speed changes.
- Seeded evaluation, failure evidence, JSON/CSV metrics, and H.264 demo output.

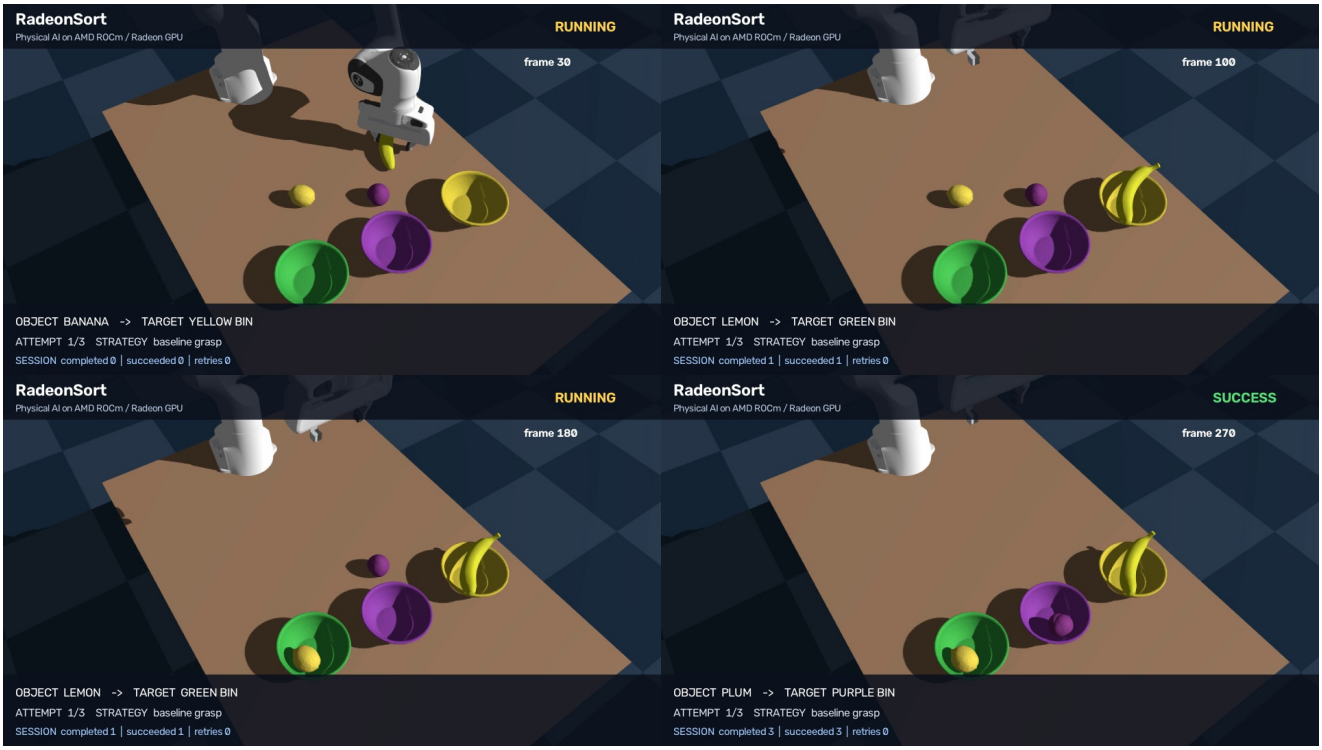

Figure 1. RadeonSort information panel and three physical target bins.

2. System Architecture and Solution Design

Figure 2. Closed-loop sorting and recovery architecture.

Task and scene configuration

The Genesis scene contains a Franka Panda arm, three YCB objects, and three colored copies of a bowl asset that act as physical bins. Task configuration provides the object class in the current version; the class determines the target bin. This report does not claim camera-based classification.

Pick, place, and verification

The controller uses object-specific grasp profiles and inverse kinematics from the Track 3 reference project. After transport and release, success is evaluated from the final object position relative to the requested target. This converts the starter procedure into a closed-loop task outcome.

Recovery state machine

Attempt	Yaw	Force	Position / Height	Transport
1	Baseline	1.00x	Adaptive center	Default; lemon slower
2	+15 deg	1.25x	+0.015 m x; lemon -0.01 m z	Reduced for lemon
3	-15 deg	1.50x	-0.015 m x, +0.015 m y; lemon +0.01 m z	Reduced for lemon

3. AMD Radeon GPU and ROCm Utilization

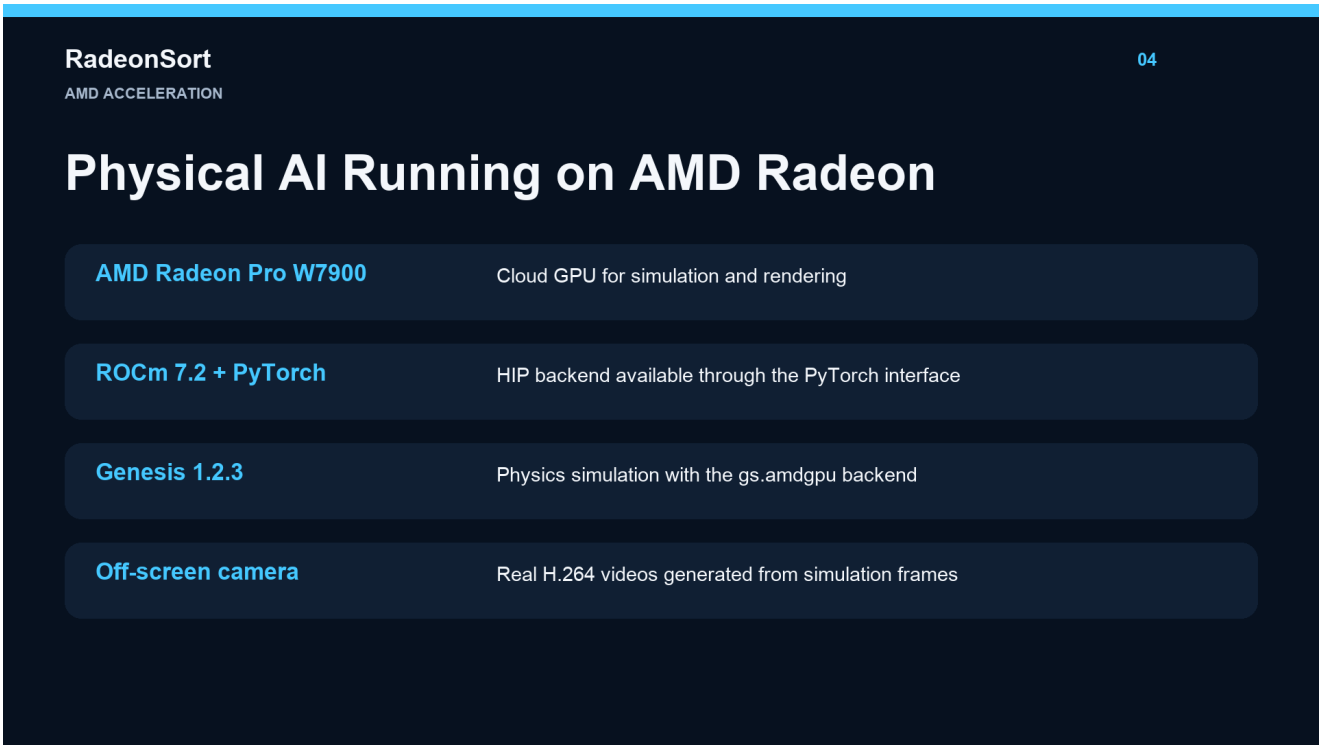

Figure 3. Runtime stack used for the project.

Runtime evidence

The cloud runtime used PyTorch 2.9.1 with ROCm/HIP 7.2 and Genesis 1.2.3. PyTorch reported GPU availability as true and identified the device as AMD Radeon Graphics. Genesis printed that it was running on AMD Radeon Graphics with backend `gs.amdgpu` and 47.98 GB device memory.

GPU workload

- Genesis physics scene construction and simulation kernels.
- Rigid-body dynamics and robot-object contact simulation.
- World-camera rendering for every recorded simulation frame.
- Repeated randomized evaluation of manipulation outcomes.

Why Radeon acceleration matters

Manipulation evaluation requires many physics steps and rendered frames for each attempt. Radeon acceleration makes it practical to run repeated trials, record failure evidence, and compare recovery policies in the same cloud environment. The demonstration video includes the real terminal command and runtime log rather than presenting only an edited simulation clip.

No trained model claim

RadeonSort currently uses task-configured class labels and scripted control; it does not claim a trained perception or visuomotor model. ROCm is used for GPU-accelerated physics and rendering in this version. Future work will add visual perception and dynamic interception.

4. Technical Contributions

4.1 Multi-class physical routing

The reference bowl is expanded into three colored, physically bounded bins. Each object class maps to a distinct target, allowing the system to demonstrate sorting rather than a single fixed pick-and-place routine.

4.2 Closed-loop failure verification

Success is not inferred from completion of the motion sequence. The final object state is checked against the requested target, and failure is recorded explicitly.

4.3 Adaptive recovery policy

After failure, the object is returned to a nearby recovery station and robot joint state is reset. Each retry changes grasp yaw, pickup position, closing force, and - for lemon - grasp height and transport speed. The policy is interpretable and visible in logs, CSV output, and the video panel.

4.4 High-friction soft fingertips

RadeonSort creates a separate Franka MJCF variant and raises friction on five fingertip collision pads. Official assets remain unchanged. A minimal compatibility patch allows the reference scene builder to accept an optional robot MJCF path.

4.5 Evidence-driven engineering

Failure videos showed that lemon often slipped during lateral transport rather than during initial pickup. This diagnosis motivated a slower lemon transport profile. The repository publishes seeded per-trial records instead of reporting only a selected successful demo.

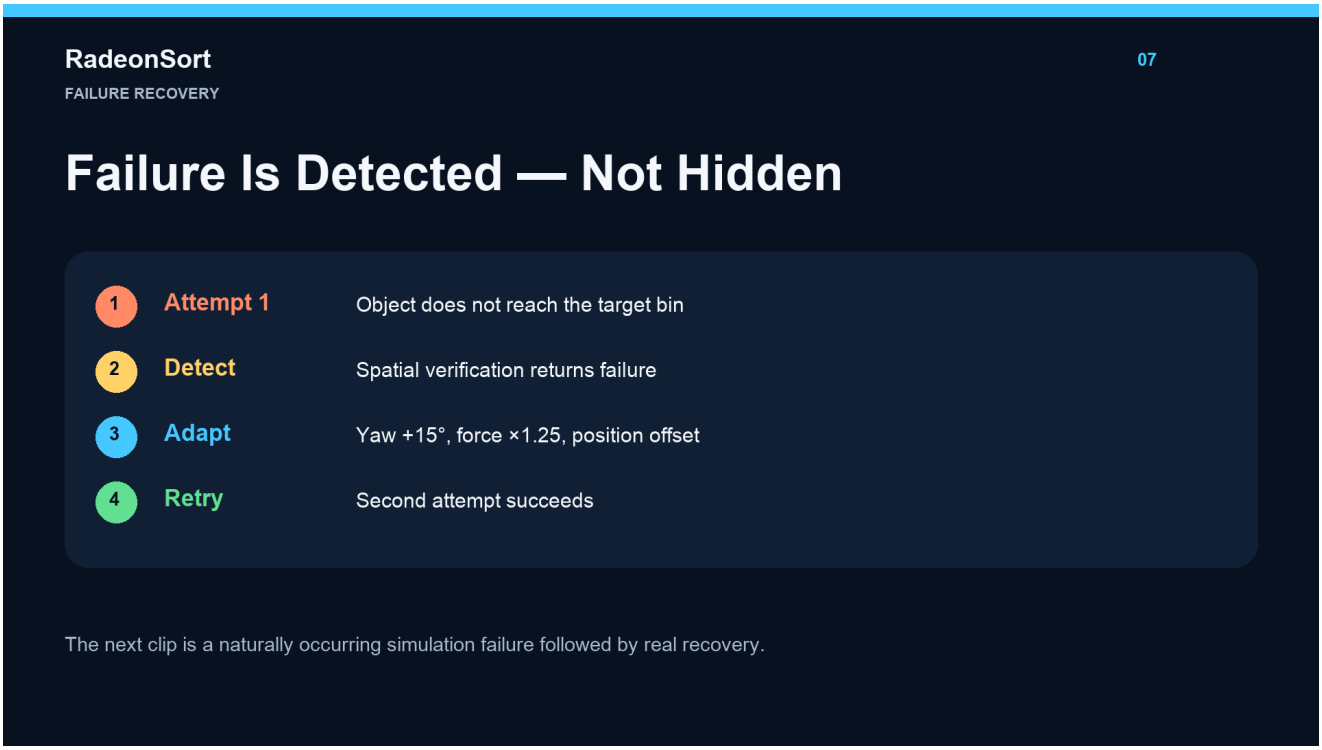

Figure 4. Naturally occurring failure followed by an interpretable recovery policy.

5. Dataset and Evaluation

Evaluation data

The evaluation uses three YCB object assets included with the Track 3 reference project: 011_banana, 014_lemon, and 018_plum. No training dataset is used because the current controller is scripted. The evaluation dataset consists of 30 seeded simulation trials, with 10 trials for each class and randomized object position and yaw. Raw records are published as JSON and CSV.

Figure 5. Reliability benchmark summary.

Object / Overall	Trials	First attempt	Final	Avg. retries	Avg. time
Banana	10	100%	100%	0.0	15.144 s
Lemon	10	70%	100%	0.4	22.827 s
Plum	10	90%	90%	0.2	17.800 s
Overall	**30**	**86.67%**	**96.67%**	**0.2**	**18.590 s**

Interpretation

Three lemon trials failed on the first attempt but were recovered, including one trial that required the third attempt. One plum trial remained unsuccessful after all attempts. The benchmark therefore demonstrates both recovery value and an honest remaining failure mode.

6. Reproducibility

Repository

Source code and raw benchmark results are available at https://github.com/520lake/RadeonSort.

Required environment

Component	Verified version / value
GPU	AMD Radeon Graphics, 47.98 GB reported memory
ROCm / HIP	7.2
PyTorch	2.9.1 ROCm build
Genesis	1.2.3
Python	3.12
Reference project	wangxunx/franka_fruit_pick_demo

Core commands

```
cd /persistent/projects/radeon-sort
bash bootstrap_cloud.sh

/workspace/rdna/bin/python radeon_sort.py --objects 011_banana 014_lemon 018_plum
--max-retries 2

/workspace/rdna/bin/python evaluate_reliability.py --trials-per-object 10 --seed
20260728
```

Generated artifacts

- radeon_sort_demo.mp4 - continuous H.264 simulation video.
- summary.json - overall task and recovery metrics.
- attempts.csv - per-attempt policy parameters and result.
- trials.csv - seeded reliability benchmark records.
- failures/ - final failure evidence for diagnosis.

7. Deliverables, Limitations, and Team

Final deliverables

Deliverable	Description
Dedicated source repository	RadeonSort implementation, bootstrap, patch, tools, and raw results
Technical report	This English PDF covering all Track 3 requirements
Demonstration video	4:15 bilingual video with cloud command, normal sorting, and recovery
Evaluation evidence	30 seeded trials, JSON/CSV records, and failure evidence

Known limitations

- Object class is supplied by task configuration; visual classification is future work.
- The submitted system is simulation-only and has not been transferred to a physical arm.
- Three YCB classes do not represent the full diversity of warehouse objects.
- A difficult plum pose remained unsuccessful after all retries in the benchmark.
- The recovery policy is interpretable but manually designed rather than learned.

Future work

The next project milestone is dynamic interception, culminating in high-speed table-tennis control. This requires moving-object state estimation, trajectory prediction, impact timing, and a higher-frequency control loop. It will be developed separately so that the stable RadeonSort V1 benchmark remains reproducible.

Team contribution

520lake is the solo participant and is responsible for project direction, Radeon Cloud setup, implementation, experiment execution, result verification, video recording, and submission. AI-assisted tools, including OpenAI Codex, were used for coding and documentation support. All submitted simulation runs, videos, and quantitative results were executed and verified by the participant.

References

[1] AMD-DEV-CONTEST, Radeon Hackathon 2026-07 official repository.

[2] wangxunx, franka_fruit_pick_demo, Track 3 reference project.

[3] Genesis Embodied AI, Genesis physics simulation platform.

[4] AMD, ROCm open software platform.

[5] Calli et al., Yale-CMU-Berkeley Object and Model Set.